



Transportation Science

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Combinatorial Optimization-Enriched Machine Learning to Solve the Dynamic Vehicle Routing Problem with Time Windows

Léo Baty, Kai Jungel, Patrick S. Klein, Axel Parmentier, Maximilian Schiffer

To cite this article:

Léo Baty, Kai Jungel, Patrick S. Klein, Axel Parmentier, Maximilian Schiffer (2024) Combinatorial Optimization-Enriched Machine Learning to Solve the Dynamic Vehicle Routing Problem with Time Windows. Transportation Science

Published online in Articles in Advance 14 Feb 2024

. <https://doi.org/10.1287/trsc.2023.0107>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2024, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Combinatorial Optimization-Enriched Machine Learning to Solve the Dynamic Vehicle Routing Problem with Time Windows

Léo Baty,^a Kai Jungel,^b Patrick S. Klein,^b Axel Parmentier,^a Maximilian Schiffer^{b,c,*}

^aCERMICS, École des Ponts, Champs sur Marne, 77455 Marne la Vallée Cedex 2, France; ^bSchool of Management, Technical University of Munich, 80333 Munich, Germany; ^cMunich Data Science Institute, Technical University of Munich, 80333 Munich, Germany

*Corresponding author

Contact: leo.baty@enpc.fr,  <https://orcid.org/0009-0001-3926-4770> (LB); kai.jungel@tum.de,  <https://orcid.org/0000-0002-6625-9942> (KJ); patrick.sean.klein@tum.de,  <https://orcid.org/0000-0002-2851-6047> (PSK); axel.parmentier@enpc.fr,  <https://orcid.org/0000-0003-1762-4947> (AP); schiffer@tum.de,  <https://orcid.org/0000-0003-2682-4975> (MS)

Received: March 30, 2023

Revised: November 18, 2023

Accepted: December 16, 2023

Published Online in Articles in Advance:
February 14, 2024

<https://doi.org/10.1287/trsc.2023.0107>

Copyright: © 2024 INFORMS

Abstract. With the rise of e-commerce and increasing customer requirements, logistics service providers face a new complexity in their daily planning, mainly due to efficiently handling same-day deliveries. Existing multistage stochastic optimization approaches that allow solving the underlying dynamic vehicle routing problem either are computationally too expensive for an application in online settings or—in the case of reinforcement learning—struggle to perform well on high-dimensional combinatorial problems. To mitigate these drawbacks, we propose a novel machine learning pipeline that incorporates a combinatorial optimization layer. We apply this general pipeline to a dynamic vehicle routing problem with dispatching waves, which was recently promoted in the EURO Meets NeurIPS Vehicle Routing Competition at NeurIPS 2022. Our methodology ranked first in this competition, outperforming all other approaches in solving the proposed dynamic vehicle routing problem. With this work, we provide a comprehensive numerical study that further highlights the efficacy and benefits of the proposed pipeline beyond the results achieved in the competition, for example, by showcasing the robustness of the encoded policy against unseen instances and scenarios.

History: This paper has been accepted for the *Transportation Science* special issue on DIMACS Implementation Challenge: Vehicle Routing Problems.

Funding: This work was supported by Deutsche Forschungsgemeinschaft [Grant 449261765].

Keywords: vehicle routing • structured learning • multistage stochastic optimization • combinatorial optimization • machine learning

1. Introduction

With the rise of e-commerce during the last decade, logistics service providers (LSPs) were exposed to increasing customer requirements, particularly with respect to (fast) delivery times. Accordingly, the concept of same-day deliveries, where LSPs guarantee to fulfill an order on the day on which they receive it, became a key element in the business-to-consumer (B2C) sector. In fact, the same-day delivery market size in the United States is expected to increase from \$5.87 billion (B) in 2019 to \$15.6B in 2024, and grew from \$7.14B to \$8.68B solely in 2021 (Placek 2022). As e-commerce and B2C deliveries remain competitive markets with several major players, retailers and LSPs continuously aim to outbid each other, which has led to continuously shortened lead times, offering delivery within as little as two hours for certain product types in selected cities (Nicolai 2016).

Realizing last-mile deliveries within such short planning horizons remains inherently challenging from an efficiency perspective as LSPs generally trade off short lead times against oversized resources, e.g., by

maintaining an oversized fleet to always be able to immediately react to incoming orders. In fact, the concept of same-day deliveries leads the concept of day-ahead planning, which has been the status quo in last-mile logistics for decades, ad absurdum. Instead of solving a combinatorially complex but static planning problem to determine cost-efficient delivery routes, LSPs have to dynamically dispatch orders to vehicles and route these vehicles in an online problem setting. Taking decisions in such a setting requires anticipating the benefit of dispatching or delaying an order while still inheriting the combinatorial complexity of the corresponding static planning problem and handling uncertainty with respect to future incoming orders.

The challenges that arise in such planning problems in practice invigorate the interest in dynamic vehicle routing problem (VRP) variants from a scientific perspective. State-of-the-art methodologies to solve such problems model the underlying planning task as a multistage stochastic optimization problem solved either directly (Pillac et al. 2013; Soeffker, Ulmer and Mattfeld 2022) or via reinforcement learning (Nazari et al. 2018;

Basso et al. 2022; Hildebrandt, Thomas and Ulmer 2023). However, both of these approaches bear a major drawback. Reinforcement-learning-based algorithms succeed in taking anticipative decisions but often struggle when being applied to high-dimensional combinatorial problems. Contrarily, stochastic optimization techniques are generally amenable to combinatorial problem settings but struggle with respect to computational efficiency in high-dimensional problems, which makes them impracticable to use in a dynamic setting (Pflug and Pichler 2014, Carpentier et al. 2015).

Against this background, we propose a new methodological approach that mitigates the aforementioned shortcomings. Specifically, we develop a machine learning (ML) pipeline with an integrated combinatorial optimization (CO) layer that allows us to efficiently solve the dynamic VRP. This pipeline mitigates the challenges of multistage stochastic optimization problems by design: its ML layer allows us to incorporate uncertainty by adequately parameterizing an instance of the underlying deterministic CO problem, which can then be efficiently solved within the CO layer. We used this pipeline in the EURO Meets NeurIPS Vehicle Routing Competition at NeurIPS 2022 (Kool et al. 2022b), where it outperformed all other approaches.

1.1. Related Work

Our work contributes to two different streams of research: from an application perspective it relates to dynamic VRPs and from a methodological perspective it relates to CO-enriched ML. We provide an overview of both related research streams in the following. For a general overview of VRPs we refer to Vidal, Laporte and Matl (2020), and for a general overview of CO-enriched ML we refer to Bengio, Lodi and Prouvost (2021) and Kotary et al. (2021).

1.1.1. Dynamic Vehicle Routing Problems. Dynamic VRPs account for the dynamic nature of real-world processes where some problem data, such as the customers to serve, their demand, or the travel time between them, are not known in advance but revealed over time. Dynamic VRPs hence have a diverse field of applications ranging from ride-hailing (Jungel et al. 2023) to grocery delivery (Fikar 2018) to emergency services (Alinaghian, Aghaie and Sabbagh 2019). To keep this literature review concise, we focus on dynamic VRPs in the context of dynamic dispatching problems in the following and refer to Pillac et al. (2013), Ulmer, Soeffker and Mattfeld (2018), and Ojeda Rios et al. (2021) for comprehensive reviews of the field. Approaches to solve dynamic VRPs can be broadly categorized into CO-based approaches, which leverage the combinatorial structure of the underlying problem, and ML-based approaches, which learn prescient policies accounting for the uncertainty of future customers.

Pure CO approaches generally adapt solution methods developed for static VRPs to the dynamic case. Specifically, these CO approaches embed (meta-)heuristics into rolling-horizon frameworks, that is, solve a static variant of the considered problem each time new information enters the system (see, e.g., Ritzinger, Puchinger and Hartl 2016; Ojeda Rios et al. 2021). Here, *myopic* approaches (see, e.g., Gendreau et al. 1999; Steever, Karwan and Murray 2019) utilize only the current problem state, whereas *look-ahead* approaches (e.g., Bent and Van Hentenryck 2004; Flatberg et al. 2007) take into account potential realizations of future periods, for example, via sampling.

ML approaches learn policies which account for the uncertainty of future observations. Accordingly, they model the underlying dynamic problem as a Markov decision process solved with either approximate dynamic programming methods (see, e.g., Ulmer, Soeffker and Mattfeld 2018), that is, policy or value function approximation, or reinforcement learning (see, e.g., Nazari et al. 2018; Kool, van Hoof and Welling 2019; Joe and Lau 2020). We refer to Raza, Sajid and Singh (2022) and Hildebrandt, Thomas and Ulmer (2023) for a review on reinforcement learning applied to dynamic VRPs.

As can be seen, various works exist that solve dynamic VRPs. Here, most approaches either utilize classical CO algorithms by sampling future scenarios or apply ML to approximate decision values which account for future expected rewards. All of these approaches contain at minimum one of the following shortcomings: classical CO-based algorithms struggle to adapt to real-time requirements of dynamic problem settings, whereas ML-based approaches often lack solution quality as they do not take the problem's combinatorial structure into account. A truly integrated approach which combines CO and ML, thus leveraging the advantage of ML in dynamic settings without disregarding the problem's combinatorial structure, has so far not been proposed for dynamic VRPs.

1.1.2. Combinatorial Optimization-Enriched Machine Learning. Many real-world combinatorial dispatching problems are subject to uncertain future events. To find combinatorial solutions which account for these uncertain events, one can integrate CO layers into ML-based pipelines. These pipelines leverage an ML layer to predict uncertain events which are then used to solve the combinatorial dispatching problem in the CO layer. Classical pipelines follow the *predict-then-optimize* paradigm and train the ML layer in isolation (Bent and Van Hentenryck 2004; Alonso-Mora, Wallar and Rus 2017; Wallar et al. 2018; Liu, He and Shen 2021). This strict separation of ML and CO layers during training, however, may result in an ML layer which does not account for the influence of the prediction error on the decision error of the downstream CO problem.

More recent approaches mitigate this issue by training the ML layer based on the decision error incurred after applying the CO layer instead of evaluating the prediction error of the ML layer in isolation (cf. Elmachtoub and Grigas 2022). This so-called *smart predict-then-optimize* framework has been applied successfully to several optimization problems, for example, to leverage travel time predictions in shortest-path and last-mile distribution problems (cf. Elmachtoub, Liang and McNellis 2020; Elmachtoub and Grigas 2022; Chu et al. 2023). This framework is, however, limited to linear cost functions in the CO layer and requires target solutions for the ML layer.

For problems with complex CO layers or when target solutions are costly to obtain, *learning by experience* techniques are often used. One example in this domain is the use of reinforcement learning techniques for (contextual) multistage stochastic optimization problems. Here, integrated ML-CO pipelines help to factorize combinatorial action spaces that are intractable for standard reinforcement learning techniques. Such approaches have been successfully applied to vehicle-dispatching problems (Tang et al. 2019, Zhou et al. 2019, Liang et al. 2022, Enders et al. 2023) or inventory management (Akkerman et al. 2023). Other, that is, non-reinforcement-learning-based, techniques in *learning by experience* settings have been applied to a wider range of problems but remain limited to nondynamic problems (Dalle et al. 2022).

A trade-off between *smart predict-then-optimize* and *learning by experience* is *learning by imitation*, which bases the learning process on target decisions of the CO layer. This allows us to train the ML layer solely on the output of the CO layer, mitigating the need for specially labeled data to train the ML layer, which would be necessary in the *smart predict-then-optimize* framework. CO-enriched ML pipelines have already been applied to combinatorial problems such as the stochastic vehicle scheduling problem (Parmentier 2021, 2022), demonstrating good performance, especially with respect to generalization on large, unseen instances. Further research in this area extended the approach to multistage optimization problems, such as the two-stage stochastic minimum spanning tree problem (Dalle et al. 2022), and online vehicle dispatching and rebalancing in dynamic autonomous mobility-on-demand systems (Jungel et al. 2023). These works, however, remain limited in several fundamental aspects. First, the downstream CO problem of their proposed CO-enriched ML pipelines is solvable in polynomial time. Second, the ML layer used in Jungel et al. (2023) is, as a generalized linear model, relatively simple and does not utilize potential performance gains promised by a deep-learning-based ML layer.

All these CO-ML approaches learn the ML predictor in an integrated way by directly backpropagating the output gradient of the CO layer to the ML layer. A major

challenge of these so-called CO-enriched ML pipelines is therefore the piecewise constant nature of most CO problems. Specifically, gradients are zero almost everywhere, and thus uninformative in such settings, rendering straightforward backpropagation ineffective. State-of-the-art methods address this issue and introduce regularization techniques that smooth CO layers to enable meaningful gradient computation, allowing their usage in ML-based pipelines. Both additive (Berthet et al. 2020) and multiplicative (Dalle et al. 2022) regularization approaches have been applied successfully to CO-enriched ML pipelines in *learning by imitation* settings with Fenchel-Young losses (Blondel, Martins and Niculae 2020).

To conclude, CO-enriched ML pipelines have been successfully utilized to solve a variety of (real-world) stochastic optimization problems, but so far either train ML layers in isolation or remain limited to relatively simple CO layers, which assumes CO layers with linear objective functions of the form $y \mapsto \theta^\top y$ where the predicted objective costs θ have the same dimension as the decision variables y .

1.2. Contributions

To close the research gap outlined above, we propose a novel ML-based pipeline enriched with a CO layer, and apply it to a novel class of dynamic VRPs introduced in the EURO Meets NeurIPS Vehicle Routing Competition that is highly interesting for academia and practice. Specifically, our work contains several contributions. From a methodological perspective, we generalize the CO layer of CO-enriched ML pipelines to a more general and potentially nonlinear class of objective functions in the CO layer. Note that in this context, we also extend the open-source library `InferOpt.jl` to such nonlinear settings. Moreover, we present the first CO-enriched ML pipeline that utilizes a metaheuristic component to solve the CO layer. By so doing, we show that our general ML-CO paradigm, which formally requires optimal solutions to derive true gradients, can work well with heuristic solutions in practice. In this context, we detail how to carefully design a metaheuristic that allows derivation of heuristic solutions, that is, approximate gradients, which enable convergence in the ML layer of our pipeline. We show how to train the ML layer of this pipeline in a supervised learning setting, that is, based on a training set derived from an anticipative strategy. We present a comprehensive numerical study to show the efficacy of our methodology in a benchmark against state-of-the-art approaches. Beyond this, our study validates that our learning approach generalizes well to unseen scenarios. Interestingly, our results point at the fact that, counterintuitive to common practice, imitating an anticipative strategy can work well for high-dimensional multistage stochastic optimization problems.

We refer to our GitHub repository (<https://github.com/tumBAIS/euro-meets-neurips-2022>) for instructions and all material necessary to reproduce the results outlined in this paper.

1.3. Outline

The remainder of this paper is organized as follows. Section 2 provides a formal definition of our problem setting before Section 3 introduces our CO-enriched ML pipeline. We then detail the individual layers of our pipeline. Specifically, Section 4 details the algorithmic framework used in the CO layer, whereas Section 5 details the learning methodology for the ML layer. Section 6 details the design of our computational study and its results. Finally, Section 7 concludes the paper.

2. Problem Setting

Our problem setting focuses on a variant of the dynamic vehicle routing problem with time windows (VRPTW) introduced in the EURO Meets NeurIPS Vehicle Routing Competition (see Kool et al. 2022b). In this dynamic VRPTW, we aim to find a cost-minimal set of routes that start and end at a central depot d and allow a fleet of vehicles to serve a set of requests $\mathcal{R}$ within a finite planning horizon $[0, T_{\max}]$. We focus on an online problem setting in which the request set $\mathcal{R}$ is initially unknown and requests continuously arrive over $[0, T_{\max}]$. Although the specifics of the request set are unknown, we assume knowledge of its distribution and refer to Section 2.6 for a critical discussion. Within this planning horizon, the fleet operator needs to serve all arriving requests and makes dispatching decisions at (equidistant) time steps $\tau \in [0, T_{\max}]$. These dispatching decisions are immutable such that vehicles cannot be rerouted to serve additional requests after being dispatched. We discretize the planning horizon into a set of n epochs $E = \{[\tau_0, \tau_1], [\tau_1, \tau_2], \dots, [\tau_{n-1}, \tau_n]\}$ and denote the start time of an epoch e as τ_e .

In each epoch, the fleet operator solves a dispatching and vehicle routing problem for the epoch-dependent request set $\mathcal{R}^e$. Each request $r \in \mathcal{R}^e$ has a certain demand q_r , and vehicles have a homogeneous vehicle capacity Q , which limits the maximum number of requests serviceable on a single route. Serving a request r takes s_r time units, and a request must be served within a request-specific time window $[\ell_r, u_r]$. Traveling from the delivery location of request i to the delivery location of another request j takes t_{ij} time units and incurs cost c_{ij} . We can straightforwardly encode an instance of an epoch's planning problem on a fully connected digraph $\mathcal{D}^e = (\mathcal{V}^e, \mathcal{A}^e)$ with a vertex set $\mathcal{V}^e = \mathcal{R}^e \cup \{d\}$, where d is the depot, and an arc set $\mathcal{A}^e$.

2.1. System State

We describe a *system state* at decision time τ_e as $x^e = \mathcal{R}^e$, where e is the epoch starting at time τ_e . Here, $\mathcal{R}^e$ contains

all requests revealed but not yet dispatched. In our specific setting no further information is required to describe the system state as there is no fleet limit. Note that we use redundant notation $x^e = \mathcal{R}^e$ to adhere to conventions commonly used in the domains of ML and vehicle routing, respectively. We can distinguish requests contained in $\mathcal{R}^e$ into two disjoint categories: *must-dispatch* requests need to be dispatched in e as dispatching these in a later epoch would violate the requests' time window upon delivery, and *postponable* requests can but do not have to be dispatched in e .

2.2. Feasible Decisions

Given the current state of the system x^e , the fleet operator chooses a subset of $\mathcal{R}^e$ that will be served by vehicles leaving the depot in this epoch, and computes the respective routes that allow for vehicle dispatching. Vehicles dispatched in epoch e leave the depot at time $\tau_e + \Delta\tau$ and their routes cannot be modified once they have been dispatched, that is, vehicles dispatched in epoch e cannot serve requests revealed in epoch e' , with $\tau_{e'} > \tau_e$. In this context, a feasible decision $y^e \in \mathcal{Y}(x^e)$ in state x^e corresponds to a set of routes that

- (i) contains all *must-dispatch* requests and
- (ii) allows each route to visit all contained requests within their respective time windows and
- (iii) for which the cumulative customer demand on each route does not exceed the vehicle capacity Q .

We can encode a feasible decision $y^e \in \mathcal{Y}(x^e)$ with a vector $(y_{i,j}^e)_{(i,j) \in \mathcal{A}^e}$ where

$$y_{i,j}^e = \begin{cases} 1 & \text{if } (i,j) \text{ is in a route of the solution} \\ 0 & \text{otherwise.} \end{cases}$$

2.3. System Evolution

The system transitions into the next epoch e' once the fleet operator decides on $y^e \in \mathcal{Y}(x^e)$. To describe $x^{e'}$, we derive $\mathcal{R}^{e'}$ by removing all requests contained in y^e from $\mathcal{R}^e$, and adding all requests that enter the system between τ_e and $\tau_{e'}$.

2.4. Policy

Let $\mathcal{X}$ denote the set of potential system states. Then, a (deterministic) policy $\pi: \mathcal{X} \rightarrow \mathcal{Y}$ is a mapping that assigns a decision $y^e \in \mathcal{Y}(x^e)$ to any system state $x^e \in \mathcal{X}$.

2.5. Objective

We aim to find a policy that minimizes the expected cost of serving all requests $\mathcal{R}$ over the planning horizon $[0, T_{\max}]$. Formally,

$$\min_{\pi} \mathbb{E} \left[\sum_{e \in E} c(\pi(x^e)) \right], \quad (1)$$

where $c: \mathcal{Y} \rightarrow \mathbb{R}$ gives the cost of routes $y \in \mathcal{Y}$.

2.6. Discussion

The problem setting defined and formalized above contains various assumptions that might be questioned from a practitioner's perspective. In particular, assuming an unlimited fleet size and full knowledge of the request distribution appears to be rather unrealistic. As this paper focuses on the methodology used to win the EURO Meets NeurIPS Vehicle Routing Competition, we decided to keep these assumptions without further questioning for the sake of consistency and reproducibility. However, we like to emphasize that the methodological pipeline presented in this paper is readily applicable to problem settings with limited fleet sizes and incomplete knowledge of the underlying request distribution as long as some historical data are available.

3. ML Pipeline with CO Layer

To explain the rationale of our CO-enriched ML pipeline, we recall that a policy π maps a system state x^e to a feasible decision y^e in $\mathcal{Y}(x^e)$. The state x^e is a set of requests $\mathcal{R}^e$, and the solution y^e is a set of feasible decisions which encode the routes covering a subset of $\mathcal{R}^e$. The set of feasible decisions $\mathcal{Y}(x^e)$ hence coincides with the set of feasible solutions of a prize-collecting VRPTW, a variant of the (static) VRPTW where it is not mandatory to serve all requests but a prize θ_j^e is collected if request j is served. In this section, we show that any decision y^e derived from an optimal policy in a dynamic problem setting corresponds to an optimal solution of a prize-collecting VRPTW for a well-chosen prize vector $\theta^e = (\theta_j^e)_{j \in \mathcal{R}^e}$. However, finding prizes θ_j^e is nontrivial, such that we resort to ML for this purpose. Specifically, we introduce a family of policies $(\pi_w)_w$ encoded by the CO-enriched ML pipeline illustrated in Figure 1: in the ML layer, a statistical model φ_w predicts θ^e based on the given system state x^e . This yields a prize-collecting VRPTW instance (x^e, θ^e) , which we solve in the CO layer with a dedicated algorithm f . The algorithm's output y^e then corresponds to our dispatching and routing decision.

In what follows, we first formally introduce the prize-collecting VRPTW and proof that we can represent every optimal decision in epoch e as a solution of a specially

constructed prize-collecting VRPTW instance. We then detail how we design our pipeline to leverage this observation for finding dispatching and routing decisions.

3.1. Prize-Collecting VRPTW

Recall that $\mathcal{D}^e = (\mathcal{V}^e, \mathcal{A}^e)$ is a fully connected digraph with vertex set $\mathcal{V}^e = \mathcal{R}^e \cup \{d\}$, comprising epoch requests $\mathcal{R}^e$ and the depot d , and that a feasible solution of the prize-collecting VRPTW $y^e \in \mathcal{Y}(x^e)$ can be encoded by the vector $(y_{i,j}^e)_{(i,j) \in \mathcal{A}^e}$ where

$$y_{i,j}^e = \begin{cases} 1 & \text{if } (i,j) \text{ is in a route of the solution} \\ 0 & \text{otherwise.} \end{cases}$$

We further consider costs $(c_{i,j})_{(i,j) \in \mathcal{A}^e}$ on each arc, and prizes $(\theta_j^e)_{j \in \mathcal{R}^e}$ on each vertex. Then, we can state the objective of the prize-collecting VRPTW as follows:

$$\max_{y \in \mathcal{Y}(x^e)} \underbrace{\sum_{\substack{(i,j) \in \mathcal{A}^e \\ j \neq d}} \theta_j^e y_{i,j}^e}_{\text{total profit}} - \underbrace{\sum_{(i,j) \in \mathcal{A}^e} c_{i,j} y_{i,j}^e}_{\text{total routing cost}}. \quad (2)$$

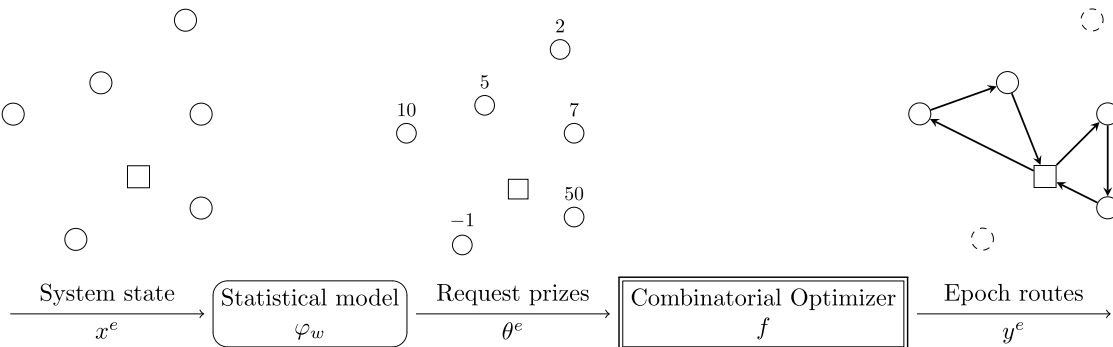
Proposition 1. For any x^e , there exists a $\theta \in \mathbb{R}^{|\mathcal{R}^e|}$ such that any optimal solution of (2) is an optimal decision with respect to (1).

Proof. Because the horizon is finite and the set of feasible decisions at each step is also finite, there exists an optimal decision y^* for x^e . Let $\tilde{\mathcal{R}}^e$ be the subset of requests of $\mathcal{R}^e$ that are dispatched in y^* . Then, any solution y which has lower or equally low routing costs and covers $\tilde{\mathcal{R}}^e$ exactly is also optimal. This follows from the Bellman equation because the routes have no impact on the evolution of the state. We can construct y by solving a prize-collecting VRPTW on $\mathcal{R}^e$ with request prizes

$$\bar{\theta}_j = \begin{cases} M & \text{if } j \in \tilde{\mathcal{R}}^e \\ -M & \text{otherwise,} \end{cases} \quad (3)$$

where $M = (|\mathcal{R}^e| \cdot \max_{(i,j) \in \mathcal{A}^e} c_{i,j})$ is a large constant. The corresponding prize-collecting VRPTW solution y clearly covers $\tilde{\mathcal{R}}^e$ exactly and has at most the routing cost of y^* . $\square$

Figure 1. Our CO-Enriched ML Pipeline



3.2. CO Layer

We embed the prize-collecting VRPTW as a *layer* in our CO-enriched ML pipeline. Hence, this CO layer must support forward and backward passes to ensure compatibility with the ML layer.

The forward pass simply solves the prize-collecting VRPTW instance defined by (x^e, θ^e) using a metaheuristic algorithm f detailed in Section 4. The backward pass backpropagates the gradient of the loss used in the learning algorithm through this layer. Section 5 introduces this loss and its gradient. To make their statement easier, we reformulate (2) as

$$f : \theta^e \mapsto \arg \max_{y \in \mathcal{Y}(x^e)} \theta^{e\top} g(y) + h(y) \quad \text{where} \\ g(y) = \left(\sum_{i \in \mathcal{V}^e} y_{i,j} \right)_{j \in \mathcal{R}^e} \quad \text{and} \quad h(y) = \sum_{(i,j) \in \mathcal{A}^e} c_{i,j} y_{i,j}. \quad (4)$$

3.3. ML Layer

Finding the optimal prizes θ^e of Proposition 1 is nontrivial. We therefore use a statistical model φ_w to predict a vector of prizes $\theta^e = \varphi_w(x^e) \in \mathbb{R}^{|\mathcal{R}^e|}$ given the system state x^e . The only technical aspect from the ML perspective is that the dimensions of the input and the output are not fixed. Indeed, the number of requests may change from one state to another and also differs across instances. We benchmark different choices for commonly used statistical models φ_w in our computational study (cf. Section 6).

3.4. Alternative ML-Based Methods

Alternative ML-based solution approaches presented in the EURO Meets NeurIPS Vehicle Routing Competition (e.g., van Doorn et al. 2022) generally proceed in two steps. They first apply a binary classifier, to decide on which requests to dispatch and postpone, respectively. They then construct routes covering these dispatched requests in a second step, essentially decoupling request dispatching from route construction. A major difficulty in this approach is to learn a statistical model which implicitly balances the current route costs and future route costs to find optimal dispatching decisions. We bypass this difficulty by taking dispatching and routing decisions simultaneously in our CO layer (4), which allows us to train our statistical model based on the routing rather than the dispatching decision.

3.5. Generalization to Other CO Problems

The ML-CO pipeline methodology presented in this section can be used for a wider range of combinatorial problems. Indeed, our methodology can more generally be applied to any problem that can be modeled as a Markov decision process. In practice, it is useful when the action space is combinatorial, which gives sense to using a CO layer. For instance, we could handle the fixed fleet

size variant of the dynamic VRPTW by adding a constraint to the feasible set $\mathcal{Y}$ of the CO layer. One can also use other CO layers in order to tackle other combinatorial and routing problems variants such as pick-up and delivery or scheduling.

4. Combinatorial Optimization Algorithm

To derive solutions of the prize-collecting VRPTW within our CO layer, we propose a metaheuristic algorithm based on hybrid genetic search (HGS). Our algorithm extends the implementation of the HGS algorithm introduced by Kool et al. (2022a), which adapts the original HGS of Vidal (2022) to the VRPTW.

Tailoring the work of Kool et al. (2022a) to our problem setting requires various modifications to support optional requests, for example, adaptations of the local search, initialization, and crossover procedures. We further introduce new mutation mechanisms, implement prize-collecting VRPTW-specific neighborhoods, and allow warm-starting of the population. In what follows, we summarize the core concepts of the HGS algorithm before detailing our prize-collecting VRPTW-specific modifications.

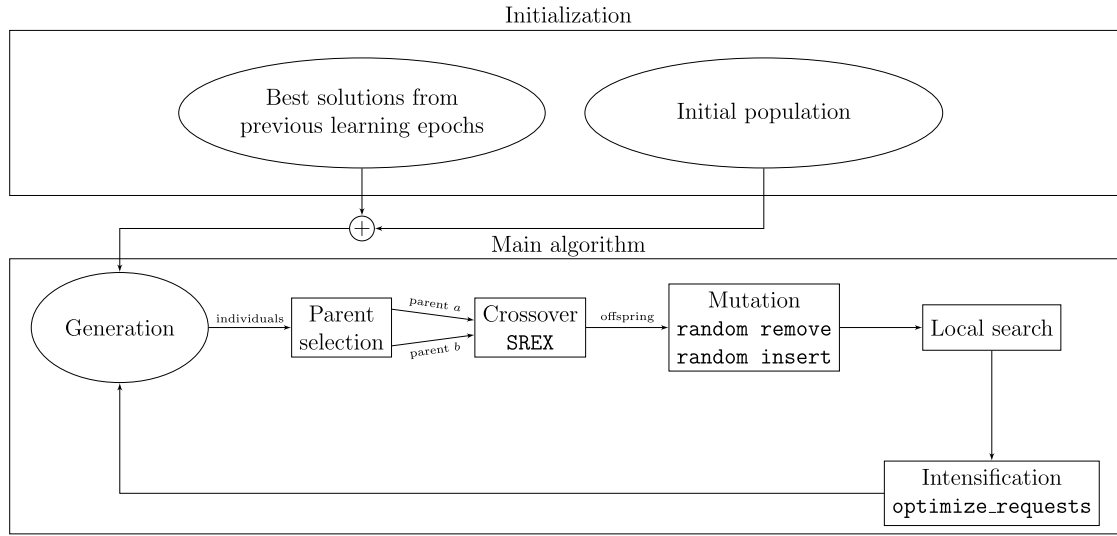
4.1. Hybrid Genetic Search for the Vehicle Routing Problem with Time Windows

The HGS algorithm is an evolutionary algorithm that maintains a *population* of solutions, organized in two disjoint subpopulations that contain feasible and infeasible solutions, respectively. The algorithm makes this population evolve over time, generating *offspring* solutions by combining promising *parent* solutions selected from the population in a randomized binary tournament. The algorithm then improves the generated offspring solution in a local search procedure. Note that it uses a penalty-based approach to explore infeasible regions of the solution space. The local search procedure yields a locally optimal solution, which is then added to the population. This may trigger a *survivor selection* procedure if the population size exceeds a certain threshold. This procedure eliminates a subset of solutions from the population. Survivor and parent selection is based on the *fitness* of a solution, a metric which captures the quality, that is, objective value, of a solution and its contribution to the population's diversity. This ensures a sufficiently diverse population, balancing diversification and intensification in the genetic algorithm. To utilize this algorithmic structure for our problem setting, that is, the prize-collecting VRPTW, we applied the following adaptations and extensions, leading to the algorithm outlined in Figure 2.

4.2. Solution Representation

We represent decisions on which requests to serve and which to ignore in our solution representation implicitly.

Figure 2. General Structure of Our Metaheuristic Algorithm



Note. Round nodes indicate populations, and square nodes indicate algorithmic components.

Specifically, we use the same giant-tour representation as in Vidal (2022) but allow incomplete giant-tours. Here, requests deemed unprofitable are absent from the giant-tour and thus not considered by traditional local search operators.

4.3. Accounting for Optional Requests During Crossover

The HGS proposed in Kool et al. (2022a) generates offspring solutions using two crossover operators: Ordered Crossover (OX) (Oliver, Smith and Holland 1987) and Selective Route Exchange (SREX) (Nagata and Kobayashi 2010). As a first modification, we remove the OX operator as our benchmarking experiments indicate that it has no substantial impact on the algorithm's performance for our problem variant. Our second modification adapts the SREX operator to the prize-collecting VRPTW. Specifically, we preserve the set of requests served by the first parent in any generated offspring by reinserting requests that are currently not served as in regular SREX, that is, one at a time.

4.4. Additional Diversification and Intensification Mechanisms

We further introduce two new mutation operators that diversify and intensify solutions based on the set of served requests. The first operator (random remove/insert), based on the result of a coin toss, either removes $\nu \cdot |R^\sigma|$ of the requests currently served or inserts $\eta \cdot |C \setminus R^\sigma|$ of the currently unserved requests. This mutation occurs with a probability of ρ right after offspring generation.

The second operator (optimize_request_set) optimizes the set of requests served by a given solution.

Specifically, this operator first removes any requests from the solution that cause a detour whose cost is higher than the request's profit and reinserts any profitable requests that are not part of the current solution. The operator perturbs insertion and removal costs with a random factor drawn uniformly from the interval $[\zeta^-, \zeta^+]$. In contrast to the first operator, we delay evaluating the second operator until the Local Search converges to avoid removing an excessive number of requests due to poor solution quality. We run this operator with probability μ . To intensify our search in regions around promising solutions, we further run this operator (without perturbation) on any solution that improves on the current best solution.

4.5. Local Search

Table 1 shows the local search operators used in our algorithm. We consider traditional and prize-collecting VRPTW-specific operators. Specifically, traditional operators refer to those defined for the VRPTW (see Kool et al. 2022a, Vidal 2022). These traditionally explore neighborhoods by exchanging arcs within a solution, that is, by changing the position of one or several requests in the current solution. Hence, they preserve the set of requests that receive service in a given solution. Prize-collecting VRPTW-specific operators, on the other hand, work with the request set exclusively, that is, they remove or insert a request from a given solution. We evaluate these after evaluating traditional VRPTW operators to avoid removing profitable requests prematurely, that is, because of bad routing decisions. As in Vidal (2022), we further distinguish between small and large neighborhood operators: small neighborhoods consider only those moves that involve requests which are geographically close and

Table 1. Local Search Operators

Type	Name	Description
Traditional (small)	relocate	Removes a single request from its route and reinserts it at a different position in the solution
	relocate pair	Removes a pair of consecutive requests from their route and reinserts them at a different position in the solution
	relocate reversed pair	Removes a pair of consecutive requests from their route and reinserts them in reverse order at a different position in the solution
	swap	Exchanges the position of two requests in the solution
	swap pair	Exchanges the positions of two pairs of consecutive requests in the solution
	swap pair with single	Exchanges the positions of a pair of consecutive requests with a single request in the solution
	2-opt	Reverses a route segment
	2-opt*	Splits two routes into two segments each, swapping a segment from the first with a segment from the second route
Traditional (large)	relocate*	Removes a request from its route and inserts it at the best possible position in any spatially overlapping route
	swap*	Removes requests a and b from routes r_a and r_b with spatial overlap, inserting a and b at the best position in r_b and r_a , respectively
Prize-collecting-VRPTW	serve request	Inserts a currently unserved request into the solution
	remove request	Removes a request from the solution

compatible w.r.t. their time windows. Large neighborhoods, on the other hand, remain unrestricted.

4.6. Initialization

We apply a preprocessing technique to account for extreme weights encountered during the learning procedure. Specifically, we maintain a set of certainly profitable and certainly unprofitable requests based on the maximum and minimum detour required, respectively. We impose constraints on our operators to avoid removing and including these profitable and unprofitable requests, respectively. We determine certainly unprofitable requests based on the following observation: if $\min_{j \in \mathcal{R}^e \cup \{d\}} c_{rj} + \min_{j \in \mathcal{R}^e \cup \{d\}} c_{rj} - \theta_r \geq \max_{i, j \in (\mathcal{R}^e \cup \{d\})^2} c_{i,j}$ holds for some request $r \in \mathcal{R}^e$, then for any route σ that includes r , a cheaper route σ' exists. To see this, let $u, v \in \mathcal{R}^e \cup \{d\}$ such that $(u, r), (r, v) \in \sigma$. Then $c_{u,r} + c_{r,v} - \theta_r \geq \max_{i, j \in (\mathcal{R}^e \cup \{d\})^2} c_{i,j} \geq c_{u,v}$, such that removing r from σ yields a cheaper route. Analogously, we force the inclusion of a request $r \in \mathcal{R}^e$ if $c_{d,r} + c_{r,d} < \theta_r$ holds. This procedure further allows us to account for must-dispatch customers during the evaluation phase by setting θ_r accordingly.

The magnitude of changes in customer prices decreases as the learning procedure converges. We exploit this behavior in our algorithm and seed the initial population, generated as in Kool et al. (2022a), with solutions of previous learning epochs. Specifically, we generate promising solutions in a new construction heuristic which first applies our `optimize_customers` operator to the solution to account for changed request prizes, and then optimizes the routing decisions using the local search procedure. Note that this is only possible while training the pipeline. When evaluating the pipeline, we use starting solutions generated as in Kool et al. (2022a) exclusively.

4.7. Discussion

Focusing on our algorithmic design decisions, we note that we have tailored our algorithm to the unique challenges of our learning methodology and training environment. Specifically, successfully computing approximate gradients that ensure convergence during training requires solutions with low variance. Here, the algorithm needs to generalize well to the different prize distributions observed during training. Beyond this, it must be capable of providing such solutions within tight time constraints to allow training within a reasonable amount of time.

We have tackled these challenges with a design that focuses on guiding the population toward promising regions of the search space through aggressive diversification (e.g., randomized insertion and deletion of requests) and intensification (`optimize_request_set`) operators, explicit request insertion and deletion neighborhoods, and warm-starting. The resulting algorithm behaves more greedily than implicit approaches (e.g., Vidal et al. 2016) but converges reliably within the tight time constraints imposed during training. We further cope with extreme prizes observed during training in a preprocessing step.

5. Learning Approach

The objective of our learning problem is to find parameters w such that the statistical model φ_w predicts a prize vector θ that leads to “good” decisions in the CO layer. To reach this objective, we train our CO-enriched ML pipeline to imitate a good policy. To do so, we follow a supervised learning setting and therefore build a data set $\mathcal{D} = \{(x^1, \bar{y}^1), \dots, (x^n, \bar{y}^n)\}$ of state instances x^i with the decisions $\bar{y}^i$ taken by the imitated policy. In this supervised learning setting, we define a loss $\mathcal{L}(\theta, \bar{y})$ which quantifies the error when we predict $f(\theta)$ instead of $\bar{y}$. Then, we formulate the learning problem as finding the

parameter $\hat{w}_D$ that minimizes the empirical risk, that is, the average loss on the training data set,

$$\hat{w}_D = \arg \min_w \sum_{i=1}^n \mathcal{L}(\varphi_w(x^i), \bar{y}^i). \quad (5)$$

The rest of this section introduces the imitated policy, the training set, the loss, and our algorithm to solve learning problem (5).

5.1. Anticipative Decisions in the Training Set

The dynamic VRPTW is difficult because future requests are unknown when taking decisions. If we knew the future, that is, if we knew all the requests at the beginning of the horizon, we would just have to solve the corresponding static VRPTW at the beginning of the horizon, and then take the corresponding dispatching decisions at each epoch. This procedure would give us the optimal anticipative policy. However, such an anticipative policy can of course not be used in practice because it relies on unavailable information. Nonetheless, we can rebuild the decisions taken by the anticipative policy a posteriori, when all information has been revealed, that is, on historical data. We therefore can learn to imitate an anticipative policy, which we gained from historical data, in our learning problem (5).

Practically, we rebuild the decision of the anticipative policy as follows. We know all the requests of an historical VRPTW instance. Then, we associate a *release time* with each request i , which is equal to the time τ_i at which the request would be revealed in a dynamic setting, thus the earliest point in time at which request i can be served. We then seek routes to serve all these requests at minimum cost while respecting the release times and the time windows. We use the adapted HGS of Kool et al. (2022a) to solve this variant of the static VRPTW. This yields a set of routes P from which we reconstruct the decisions taken in each epoch as follows. For route $p \in P$, let $\tau_p = \max_{r \in p} \tau_r$ be the latest release time of any request route p serves. The anticipative policy dispatches route p in the first epoch e where $\tau_e \geq \tau_p$, that is, the first epoch where all served requests have been revealed.

Instead, one could also derive an anticipative decision by dispatching a route at the latest possible epoch, such that the route is still feasible with respect to the requests' time windows. This may increase the degree of freedom for decisions in subsequent periods. However, in the environment studied, there is no leeway in the time windows to do so, such that early dispatching and late dispatching lead to almost identical decisions. Although we accordingly excluded such a strategy from this study, it may prove worthwhile to consider a delayed anticipative decision when working in a different problem setting.

5.2. Loss Function

We recall that the objective of our learning problem is to find parameter values w of a statistical model φ_w such

that for any state x , the CO layer predicts a good decision $y = f(\theta)$ for the prize-collecting VRPTW instance $(x, \theta = \varphi_w(x))$. More precisely, for each state-decision pair $(x, \bar{y})$ in the data set, we want the target decision $\bar{y}$ to be as close as possible to the optimal solution of the prize-collecting VRPTW (2). Hence, it is natural to take the nonoptimality of $\bar{y}$ as a solution of $f(\theta)$ as loss function:

$$\mathcal{L}(\theta, \bar{y}) = \max_{y \in \mathcal{Y}(x^e)} \{\theta^\top g(y) + h(y)\} - (\theta^\top g(\bar{y}) + h(\bar{y})), \quad (6)$$

with g and h defined in (4). Note that what follows in this subsection is valid for any CO layer of the form $f: \theta \mapsto \arg \max_y \theta^\top g(y) + h(y)$, for any g and h .

We clearly have $\mathcal{L}(\theta, \bar{y}) \geq 0$ in general, and $\mathcal{L}(\theta, \bar{y}) = 0$ only if y is an optimal solution of (4). Unfortunately, the polyhedron $\mathcal{P}(\bar{y}) = \{\theta \in \mathbb{R}^{|x|} \mid \theta^\top g(\bar{y}) + h(\bar{y}) \geq \theta^\top g(y) + h(y), \forall y \in \mathcal{Y}(x)\}$ is generally highly degenerate. For instance, if $h = 0$, then $\theta = 0$ belongs to $\mathcal{P}(\bar{y})$ for all y in $\mathcal{Y}$. This implies that $\theta = 0$ would be an optimum of our learning problem, which is a problem because such a θ allows our CO layer to return any solution in $\mathcal{Y}(x)$. This degeneracy can be removed by considering the perturbed loss

$$\mathcal{L}_\varepsilon(\theta, \bar{y}) = \mathbb{E} \left[\max_{y \in \mathcal{Y}(x)} \{(\theta + \varepsilon Z)^\top g(y) + h(y)\} \right] - (\theta^\top g(\bar{y}) + h(\bar{y})), \quad (7)$$

where $\varepsilon > 0$ and $Z \sim \mathcal{N}(0, \mathbf{I}_{|x|})$ is a Gaussian perturbation. The perturbed loss $\mathcal{L}_\varepsilon$ has been considered in the literature only in the case where $g(y) = y$ and $h(y) = 0$ (cf. Berthet et al. 2020). Because of nonzero h , the geometry of the loss changes. Notably, although the size of ε does not matter when $h = 0$ (cf. Parmentier and T'Kindt 2021), it does matter when h is nonzero: the larger ε , the smaller the impact of h on the prediction. Proposition 2 summarizes the geometry of these losses in our more general context.

Proposition 2. *Let $x \in \mathcal{X}$, $\bar{y} \in \mathcal{Y}(x)$. Let $\mathcal{C}(\bar{y}) = \{\theta \in \mathbb{R}^{|x|} \mid \theta^\top g(\bar{y}) \geq \theta^\top g(y), \forall y \in \mathcal{Y}(x)\}$ be the normal cone associated with $g(\bar{y})$.*

1. *The enumeration is part of the proposition and the items represent mathematical arguments. Therefore we suggest to keep it as is.*

$$\underbrace{g \left(\arg \max_{y \in \mathcal{Y}(x)} \theta^\top g(y) + h(y) \right)}_{=g(f(\theta))} - g(\bar{y}) \in \partial_\theta \mathcal{L}(\theta, \bar{y}).$$

2. $\theta \mapsto \mathcal{L}_\varepsilon(\theta, \bar{y})$ is C^∞ and convex with gradient

$$\begin{aligned} \nabla_\theta \mathcal{L}_\varepsilon(\theta, \bar{y}) &= \mathbb{E} \left[g \left(\arg \max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top g(y) + h(y) \right) \right] - g(\bar{y}) \\ &= \mathbb{E}[g(f(\theta + \varepsilon Z))] - g(\bar{y}). \end{aligned}$$

3. $\mathcal{L}_\varepsilon(\theta, \bar{y}) \geq \mathcal{L}(\theta, \bar{y})$.

4. $\mathcal{C}(\bar{y})$ is the recession cone of $\mathcal{P}(\bar{y})$.

5. Let $\theta \in \mathbb{R}^{|x|}$. If η is in $\mathcal{C}(\bar{y}) \setminus \{0\}$, then $\lambda \mapsto \mathcal{L}(\theta + \lambda\eta, \bar{y})$ is nonincreasing. If in addition $\mathcal{C}(\bar{y}) \neq \mathbb{R}^{|x|}$, then $\lambda \mapsto \mathcal{L}_\varepsilon(\theta + \lambda\eta, \bar{y})$ is decreasing.

6. Let $\theta \in \mathbb{R}^{|x|}$. If η is in the interior $\mathring{\mathcal{C}}(\bar{y})$ of $\mathcal{C}(\bar{y})$, then $\lim_{\lambda \rightarrow \infty} \mathcal{L}(\theta + \lambda\eta, \bar{y}) = \lim_{\lambda \rightarrow \infty} \mathcal{L}_\varepsilon(\theta + \lambda\eta, \bar{y}) = 0$.

Proof.

1. $\theta \mapsto \mathcal{L}(\theta, \bar{y})$ is linear and convex as it is the maximum of mappings that are linear in θ . Let $(\theta, \tilde{\theta}) \in \mathbb{R}^{|x|} \times \mathbb{R}^{|x|}$. Let y^* be in $\arg \max_{y \in \mathcal{Y}(x)} \theta^\top g(y) + h(y)$ and $\tilde{y}$ in $\arg \max_{y \in \mathcal{Y}(x)} \tilde{\theta}^\top g(y) + h(y)$. By definition of $\tilde{y}$, we have $\tilde{\theta}^\top g(\tilde{y}) + h(\tilde{y}) \geq \tilde{\theta}^\top g(y^*) + h(y^*)$, which gives $\mathcal{L}(\tilde{\theta}, \bar{y}) \geq \mathcal{L}(\theta^*, \bar{y}) + (\tilde{\theta} - \theta^*)^\top (g(y^*) - g(\bar{y}))$ and the subgradient in $\partial_\theta \mathcal{L}(\theta, \bar{y})$.

2. Because $\mathcal{L}_\varepsilon(\theta, y) = \mathbb{E}[\mathcal{L}(\theta + \varepsilon Z, y)]$, we obtain that $\theta \mapsto \mathcal{L}_\varepsilon(\theta, y)$ is C^∞ as a convolution product of $\theta \mapsto \mathcal{L}(\theta, \bar{y})$ with a Gaussian density (which is C^∞) and is convex as an expectation of a convex function. The gradient follows from the subgradient of $\mathcal{L}(\theta, y)$.

3. Let $\theta \in \mathbb{R}^{|x|}$. For all $\tilde{y}$ in $\mathcal{Y}(x)$, we have

$$\begin{aligned} & \text{for all } z \in \mathbb{R}^{|x|}, \quad \max_y (\theta + \varepsilon z)^\top g(y) + h(y) \\ & \geq (\theta + \varepsilon Z)^\top g(\tilde{y}) + h(\tilde{y}) \end{aligned}$$

$$\begin{aligned} \text{hence, } & \mathbb{E} \left[\max_y (\theta + \varepsilon Z)^\top g(y) + h(y) \right] \\ & \geq \theta^\top g(\tilde{y}) + h(\tilde{y}) \quad (\text{since } \mathbb{E}[Z] = 0) \end{aligned}$$

$$\begin{aligned} \text{and, } & \mathbb{E} \left[\max_y (\theta + \varepsilon Z)^\top g(y) + h(y) \right] - (\theta^\top g(\bar{y}) + h(\bar{y})) \\ & \geq \theta^\top g(\tilde{y}) + h(\tilde{y}) - (\theta^\top g(\bar{y}) + h(\bar{y})). \end{aligned}$$

Finally, $\mathbb{E}[\max_y (\theta + \varepsilon Z)^\top g(y) + h(y)] - (\theta^\top g(\bar{y}) + h(\bar{y})) \geq \max_y \{\theta^\top g(y) + h(y)\} - (\theta^\top g(\bar{y}) + h(\bar{y}))$, that is, $\mathcal{L}_\varepsilon(\theta, \bar{y}) \geq \mathcal{L}(\theta, \bar{y})$.

4. Let $\theta \in \mathcal{P}(\bar{y})$. We have $\forall y, (\theta + \lambda\eta)^\top g(\bar{y}) + h(\bar{y}) \geq (\theta + \lambda\eta)^\top g(y) + h(y) \Leftrightarrow \forall y, \theta^\top g(\bar{y}) + h(\bar{y}) + \lambda\eta^\top (g(\bar{y}) - g(y)) \geq \theta^\top g(y) + h(y)$. If η is in $\mathcal{C}(\bar{y})$, then $\eta^\top (g(\bar{y}) - g(y)) \geq 0$, and $\theta + \lambda\eta$ is in $\mathcal{P}(\bar{y})$ for any $\lambda > 0$. If η is not in $\mathcal{C}(\bar{y})$, then $\eta^\top (g(\bar{y}) - g(y)) < 0$ and there exists a $\lambda > 0$ such that $\theta + \lambda\eta$ is not in $\mathcal{P}(\bar{y})$.

5. Let $\eta \in \mathcal{C}(\bar{y})$. Let us denote by $\hat{y} = \arg \max_y (\theta + \lambda\eta)^\top g(y) + h(y)$. We have

$$\begin{aligned} & \text{for all } \lambda, \quad \mathcal{L}(\theta + \lambda\eta, \bar{y}) \\ & = (\theta + \lambda\eta)^\top g(\hat{y}) + h(\hat{y}) - [(\theta + \lambda\eta)^\top g(\bar{y}) + h(\bar{y})] \end{aligned}$$

By taking the derivative with respect to λ , we obtain $\eta^\top (g(\hat{y}) - g(\bar{y}))$, which is negative by definition of η . This gives us the nonincreasing property of $\lambda \mapsto \mathcal{L}(\theta + \lambda\eta, \bar{y})$.

Similarly, by denoting $\hat{y}_\varepsilon(Z) = \arg \max_y (\theta + \lambda\eta + \varepsilon Z)^\top g(y) + h(y)$ for all Z , we obtain the derivative of $\lambda \mapsto \mathcal{L}_\varepsilon(\theta + \lambda\eta, \bar{y})$: $\eta^\top (\mathbb{E}[g(\hat{y}_\varepsilon(Z))] - g(\bar{y}))$. If $\mathcal{C}(\bar{y}) \neq \mathbb{R}^{|x|}$, then $\mathbb{P}(\theta + \lambda\eta + \varepsilon Z \notin \mathcal{C}(\bar{y})) > 0$. Hence, $\mathbb{E}[g(\hat{y}_\varepsilon(Z))] \neq g(\bar{y})$, $\eta^\top (\mathbb{E}[g(\hat{y}_\varepsilon(Z))] - g(\bar{y})) < 0$, and therefore $\lambda \mapsto \mathcal{L}_\varepsilon(\theta + \lambda\eta, \bar{y})$ is decreasing.

6. Let $\eta \in \mathring{\mathcal{C}}(\bar{y})$. By definition of $\mathring{\mathcal{C}}(\bar{y})$, we have $\eta^\top (\bar{y} - y) > 0$ for all $y \neq \bar{y}$. Hence, there exists $M > 0$ such that for all $\lambda \geq M$ and $y \neq \bar{y}$, we have $(\theta + \lambda\eta)^\top \bar{y} + \geq (\theta + \lambda\eta)^\top y$. That is, for all $\lambda \geq M$ we have $\theta + \lambda\eta \in \mathcal{P}(\bar{y})$, that is, $\mathcal{L}(\theta + \lambda\eta, \bar{y}) = 0$. Hence, $\lim_{\lambda \rightarrow \infty} \mathcal{L}(\theta + \lambda\eta, \bar{y}) = 0$.

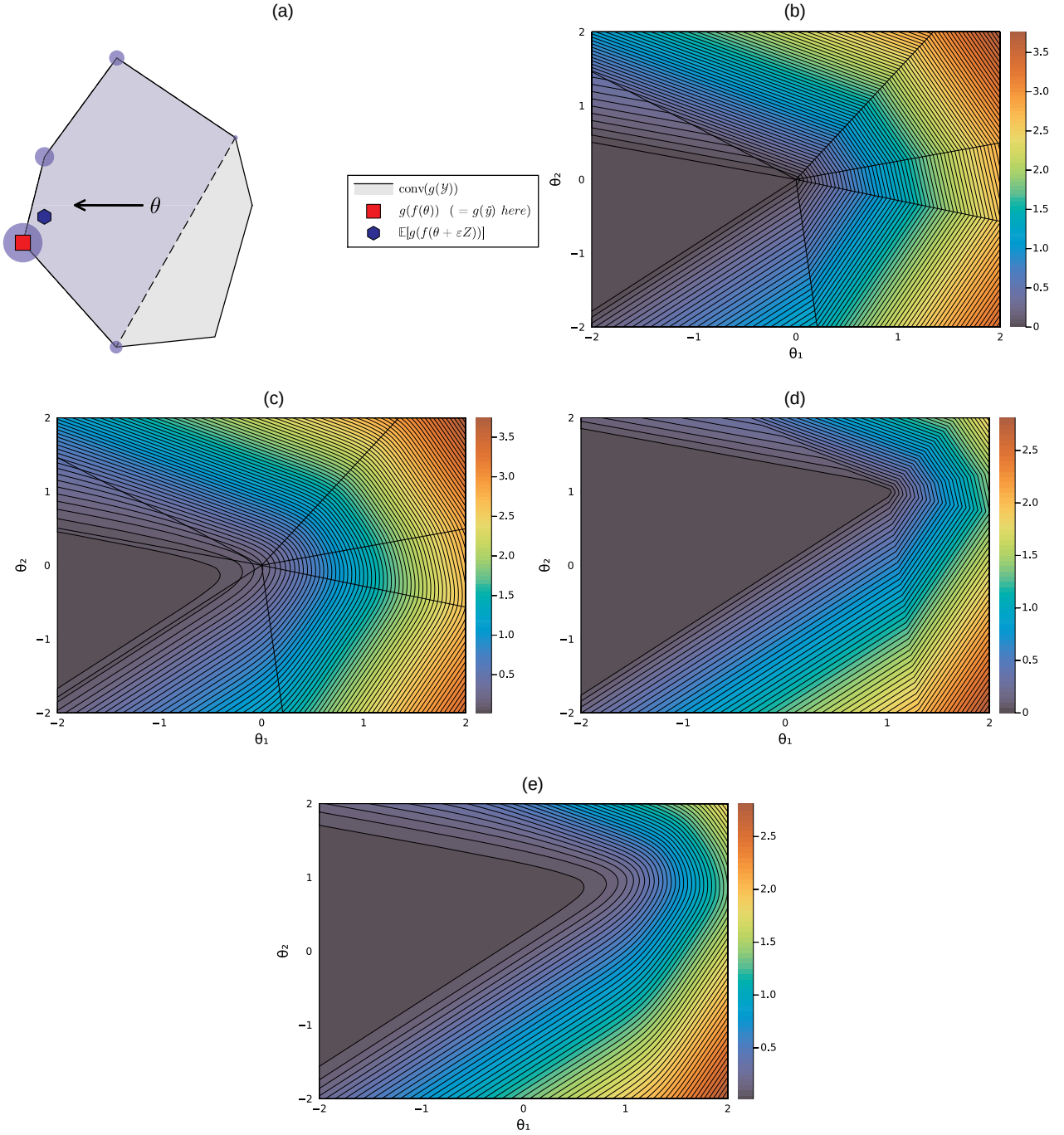
If $\mathcal{C}(\bar{y}) \neq \mathbb{R}^{|x|}$, from point 5 and the previous limit, we get that $\lambda \mapsto \mathcal{L}(\theta + \varepsilon z + \lambda\eta, \bar{y})$ monotonically decreases to zero for any z . The monotone convergence theorem therefore gives $\lim_{\lambda \rightarrow \infty} \mathcal{L}_\varepsilon(\theta + \lambda\eta, \bar{y}) = 0$. If $\mathcal{C}(\bar{y}) = \mathbb{R}^{|x|}$, the result is immediate. $\square$

The perturbed loss $\mathcal{L}_\varepsilon$ has properties that make it very suitable for a learning problem. Indeed, it is convex and smooth, and although the expectation in its gradient is intractable, sampling Z gives a stochastic gradient. The learning problem (5) with loss $\mathcal{L}_\varepsilon$ can therefore be solved using stochastic gradient descent. Furthermore, $\theta \mapsto \mathcal{L}_\varepsilon(\theta, \bar{y})$ tends to zero only if θ is far in the interior of $\mathcal{C}(\bar{y})$, which means that there is no ambiguity on the fact that $\bar{y}$ is an optimal solution of (4) for θ , and removes the degeneracy issue of $\theta \mapsto \mathcal{L}(\theta, \bar{y})$. This situation is illustrated in two dimensions in Figure 3. Figure 3(a) represents the polytope corresponding to the convex envelope $\text{Conv}(g(\mathcal{Y})) = \text{Conv}(\{g(y), \forall y \in \mathcal{Y}\})$. The algorithm necessarily outputs a vertex of this polytope (red square) as the objective function of the CO layer is linear in $g(y)$. The dark blue hexagon is the output $\mathbb{E}[g(f(\theta + \varepsilon Z))]$ of the perturbed CO layer, which can be seen as the expectation over a distribution (blue circles) on the vertices of the polytope. On Figure 3(b), with $h = 0$ and no perturbation, we can see that the loss $\mathcal{L}(\theta, \bar{y})$ is zero for any θ in the normal cone of $\bar{y}$ (with $g(\bar{y})$ being the red square), and that the origin belongs to the normal cone of any vertex y . On Figure 3(c), the perturbation has been added and we can see that $\mathcal{L}(\theta, \bar{y})$ goes to zero only when we are safely inside the cone, which means that the output of the combinatorial optimization layer is nonambiguous. Finally, on Figure 3, (d) and (e), we can see that a nonzero h changes the geometry for small θ values but not for large θ values.

5.2.1. Nonoptimal CO Layer. Note that our CO layer f is a (meta-)heuristic algorithm and thus does not guarantee an optimal solution, which is not in line with the assumptions of the theory reviewed in this section. However, we observe that this is not a problem in practice as f outputs solutions close enough to optimal ones.

5.2.2. Generalization to Other CO Problems. As mentioned at the beginning of this subsection, this learning approach works for any CO layer of the form (4) for any g, h , and feasible set $\mathcal{Y}$. Additionally, at the end of Section 3 we explain that we can build a pipeline for any problem modeled as a Markov decision process, the only requirement in practice being able to compute $f(\theta)$. However, for being able to learn this pipeline, we need an additional

Figure 3. (Color online) Visualization of the Loss for Two-Dimensional θ



Notes. Contour plots of the loss $\mathcal{L}(\cdot, \bar{y})$ and its perturbed version $\mathcal{L}_\epsilon(\cdot, \bar{y})$ for two different values of h and its respective polytope (a). Thick lines on panels (b) and (c) represent the normal fan associated with the polytope. We can see the perturbed regularization “pushing” the loss inside the normal cone $\mathcal{C}(\bar{y})$. (a) Polytope $\text{Conv}(g(Y))$; (b) $\theta \mapsto \mathcal{L}(\theta, \bar{y})$ for $h = 0$; (c) $\theta \mapsto \mathcal{L}_\epsilon(\theta, \bar{y})$ for $h = 0$; (d) $\theta \mapsto \mathcal{L}(\theta, \bar{y})$ for $h \neq 0$; (e) $\theta \mapsto \mathcal{L}_\epsilon(\theta, \bar{y})$ for $h \neq 0$.

assumption: the dynamic of the system needs to be of the form $x_{t+1} = f(x_t, y_t, \psi_t)$, with ψ_t an exogenous noise independent of decision y_t . This is needed in order to derive the anticipative policy to imitate.

5.2.3. Perturbed Loss and Fenchel-Young Loss. The perturbed loss $\mathcal{L}_\epsilon$ defined in (7) is a generalized version of the Fenchel-Young loss defined in Berthet et al. (2020).

It has a missing term $C(\hat{y})$, a constant depending on the target solution only and therefore not affecting the gradient with respect to θ .

6. Computational Study

The aim of our computational study is twofold. First, we validate the performance of our CO-enriched ML

pipeline in a benchmark against several state-of-the-art approaches. Second, we conduct extensive numerical experiments to assess the impact of different training settings on the performance of our CO-enriched ML pipeline. Specifically, we investigate the impact of (i) the feature set, (ii) the size of instances in the training set, (iii) the training set size, (iv) the imitated target strategy, and (v) the type of statistical model used.

For this purpose, we first detail the design of our computational study in Section 6.1, and the different benchmarks in Section 6.2. We then specify the study setup in Section 6.3, and present the results of our benchmark study in Section 6.4. Finally, we discuss the results of our experiments on different training settings in Section 6.5.

6.1. Design of Experiments

We design our computational study in line with the problem setting presented in the EURO Meets NeurIPS Vehicle Routing Competition (Kool et al. 2022b). We consider a set of VRPTW instances derived from real-world data of a United States-based grocery delivery service. We refer to these instances as *static* instances. A static instance contains a set of requests, each with a specific location, a service time, a demand, and a time window. Hence, each instance implicitly defines a distribution of request locations, service times, demands, and time windows. We generate an instance of the *dynamic* problem from a static instance as follows: we first discretize the planning horizon of the static instance, which is between five and nine hours, into one-hour epochs. Then, for each epoch e , we sample m requests randomly, which are constructed by drawing from the static instance's location, demand, service time, and time window distributions independently. We refer to m as the *sample size* in the remainder of this section. We discard requests that are infeasible with respect to the starting time of the current epoch and the chosen time window. The remaining requests form the set of requests revealed in epoch e . This sampling approach has two implications. First, the expected number of requests arriving in each epoch decreases with advancing epochs as the probability of sampling a feasible time window decreases. Second, time windows that end closer to the end of the planning horizon are more likely to appear, as these time windows have a higher probability to be feasible. In line with the challenge's problem setting, we minimize travel time only, such that the number of vehicles, service times, and waiting times are not part of the objective. Furthermore, we do not limit the number of vehicles and assume knowledge over the static instance's request distribution. Finally, we note that we can generate several dynamic instances from one static instance by varying the seed used to sample from the request distribution. In what follows, we refer to this seed as the *instance seed*.

Although we already discussed the shortcomings of the problem setting in Section 2, a few comments on the

problem setting, especially with respect to the epoch length and unlimited vehicle fleet, are in order. When interpreting our results, one should be aware of the fact that the epoch length of one hour limits the responsiveness of the system to new requests. However, our methodology is not dependent on the epoch length. We can vary the epoch length between a dynamic epoch length, which defines the time between two requests entering the system, and a rather long static epoch length, as considered in this experimental design, as long as we can state the underlying problem as a Markov decision process. A dynamic short epoch length leads to an immediate response to new requests when entering the system. The unlimited vehicle fleet size is also no methodological limitation in practice, as we can introduce a limited fleet size by adapting the constraints in the CO layer. In parallel works, we provide evidence that our methodology not only is in theory applicable to such settings but also provides a competitive performance. For details, we refer the interested reader to Jungel et al. (2023), where we study a mobility-on-demand problem setting with a limited fleet size in which we use a point process to model requests entering the system, leading to a short response time of one minute. In this setting, our methodological paradigm improves upon existing benchmarks and allows us to provide a new state-of-the-art.

6.2. Benchmark Policies

We evaluate the performance of our CO-enriched ML pipeline against a set of benchmark policies. These policies follow a two-stage approach by first deciding on the set of requests to dispatch and then forming covering routes in a second step by solving a VRPTW on the dispatched requests using the HGS algorithm (cf. Kool et al. 2022a, Vidal 2022). For each benchmark policy, we allow a time limit for finding decisions of 90 seconds per epoch, if not stated differently. In what follows, we briefly introduce the benchmark policies considered in this computational study.

Greedy policy: The greedy policy dispatches all requests as soon as they enter the system.

Lazy policy: The lazy policy delays dispatching a request as long as possible, that is, to the epoch where it becomes must-dispatch.

Random policy: The random policy dispatches postponable requests with a probability of 50%. It further dispatches all must-dispatch requests.

Rolling-horizon policy: The rolling-horizon policy samples a scenario for the remaining epochs and applies the HGS in a similar way as the anticipative strategy assuming that the sampled scenario represents the true scenario. This yields a set of routes based on which it decides which requests to dispatch. Specifically, it dispatches a request of the current epoch if and only if that request shares a route with a must-dispatch request of the current epoch. We assign a time limit of

600 seconds to ensure convergence of the HGS because this policy entails solving a VRPTW on a complete scenario. Note that this extends the 90-second time limit allowed to the other benchmark policies by 510 seconds.

Monte Carlo policy: The Monte Carlo policy samples nine scenarios for the remaining epochs, solved individually as in the rolling-horizon policy. It dispatches a request based on a majority decision, that is, if and only if the rolling-horizon policy dispatches the request in at least 50% of the sampled scenarios. We raise the time limit accordingly, that is, allow a total of 5,400 seconds.

ML-CO policy: The ML-CO policy is encoded in the CO-enriched ML pipeline introduced in Section 3. Unless specified otherwise, we train our ML-CO policy on a set of 15 training instances using a sample size of 50 requests per epoch, derived using the anticipative strategy as detailed in Section 5.1. Note that we train on some instances and generalize on others to stick to the challenge practice. Note that a firm that would use such a policy in practice may have a single instance corresponding to a city. It would use a training set with some scenarios and generalize on a test set with different scenarios. Such a setting enables us to overfit the instance and get performance on it. We limit the runtime of the HGS used to derive the anticipative solutions to 3,600 seconds. Our ML layer uses a feedforward neural network with four hidden layers, each comprising 10 neurons. For the ML-CO policy, we set the time limit for finding decisions to 90 seconds per epoch. Table 2 indicates that this time limit does not impact the performance of our ML-CO policy significantly.

We note that the *greedy*, *lazy*, and *random* policies are the baseline heuristics used in the EURO Meets NeurIPS Vehicle Routing Competition and refer the interested reader to Kool et al. (2022b) for more details.

We additionally include the anticipative strategy used during training (cf. Section 5.1) as a baseline. To derive the anticipative baseline, we solve the offline VRPTW using the HGS algorithm proposed in Kool et al. (2022a) with a time limit of 3,600 seconds. Note that the HGS algorithm is a heuristic, which implies that it is possible to find a solution which outperforms the anticipative baseline.

6.3. Study Setup

We evaluate the performance of the benchmark policies on a total of 500 dynamic instances generated from 25 different request distributions, that is, static instances

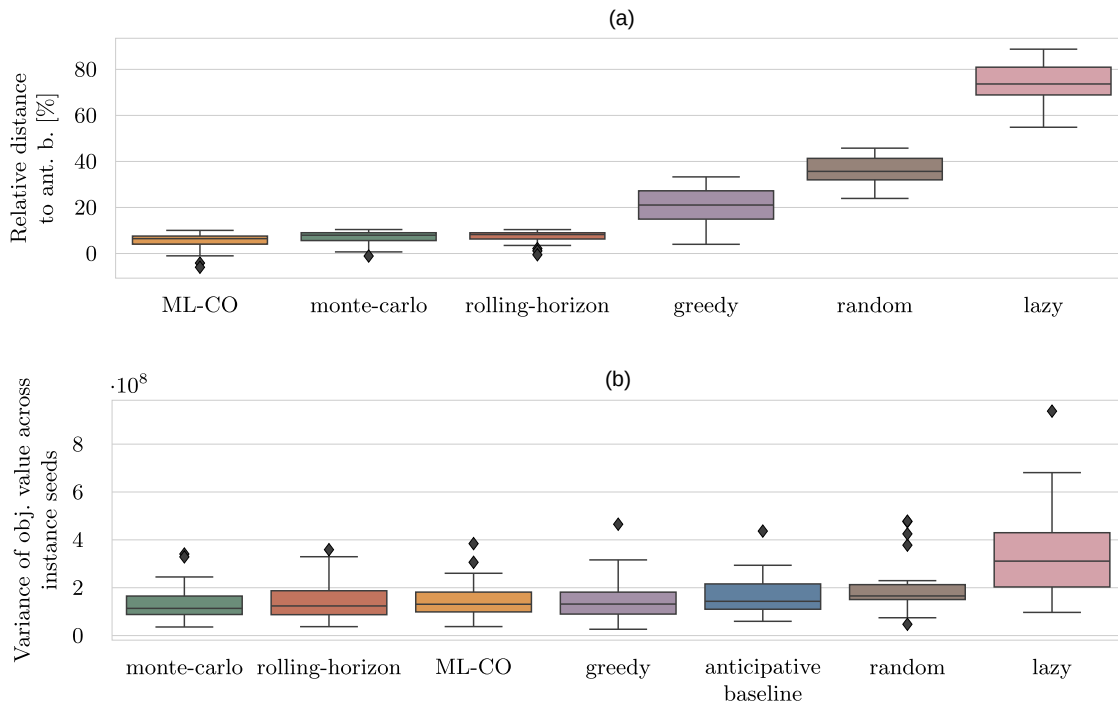
(cf. Section 6.1). Our results are based on aggregated objective values for each considered static instance, that is, the average over the 20 different dynamic instances generated from each request distribution. We generate dynamic instances as detailed in Section 6.1 using a sample size of $m = 100$ requests per epoch. We note that evaluation instances do not correspond to those used to train individual models and particularly point out that we, unless noted otherwise, generate training instances for the ML-CO policy using a smaller sample size ($m = 50$). All experiments limit the runtime of the ML-CO policy to 90 seconds per epoch.

6.4. Performance Analysis

Figure 4(a) compares the performances of the introduced benchmarks. The figure shows the gap of the policy's objective value relative to the anticipative baseline (ant. b.). In general, we can see that benchmark policies which consider information about the uncertain appearance of future requests, that is, rolling-horizon, Monte Carlo, and ML-CO policies, and the anticipative baseline, outperform benchmarks which make dispatching decisions based on the current epoch only, that is, lazy, random, and greedy. Specifically, these perform on average 74.05%, 36.29%, and 20.99% worse than the anticipative baseline, respectively. The *rolling-horizon* policy performs 7.12% worse than the anticipative baseline. We see that considering the future impact of our dispatching decision based on only a single scenario already improves the performance in comparison with the greedy policy significantly. This is surprising as the scenario drawn is often only a poor representation of the requests actually observed in later epochs because of the size of the sample space. However, the distance between the drawn scenario and the correct scenario might be outweighed by the benefit of combining future requests with actual requests in low-cost routes. The *Monte Carlo* policy outperforms the rolling-horizon policy and has a gap of only 6.97% to the anticipative baseline. This shows that a better approximation of future uncertainties, achieved through a higher number of sampled scenarios, leads to better dispatching decisions, thus improving performance. Yet, the improvement with respect to the rolling-horizon benchmark is rather small in comparison with the improvement from the greedy policy to the rolling-horizon policy. Our *ML-CO* policy outperforms all other online policies and is only 5.15% worse than the anticipative baseline. This is

Table 2. Performance of Our ML-CO Policy for Different Time Limits That We Allow for the Prize-Collecting-HGS During Evaluation

	Decision time limit during evaluation (seconds)					
	30	60	90	120	180	240
Relative distance to anticipative baseline	5.66%	5.18%	5.15%	5.01%	4.89%	4.87%

Figure 4. (Color online) Performance of Benchmark Policies

Notes. (a) Performance relative to anticipative baseline solution. (b) Variance of objective value with respect to instance seeds.

surprising as the general consensus in literature on multistage stochastic problems indicates the contrary, that is, that imitating an anticipative strategy does not generalize well. In our extended analysis we point to some signals in our results which indicate that one may achieve improved performance by imitating a better policy.

Result 1. Our ML-CO policy performs best across all online benchmark policies and outperforms the Monte Carlo policy by 1.57%. This indicates that learning a policy by imitating an anticipative strategy yields good performances in this problem setting.

Figure 4(b) shows the variance of the objective value over several instance seeds for the considered benchmark policies. Comparing Figure 4, (a) and (b), shows a clear trend: policies with low variance outperform policies with high variance. This trend results from finding more robust solutions that generalize well over uncertain future observations. Note that, although performing worse, the Monte Carlo policy has less variance than our ML-CO policy. This relates to the sampling approach the Monte Carlo policy is based on, which yields a robust solution that performs well over all sampled scenarios and therefore focuses too much on variance minimization. Our ML-CO policy, on the other hand, balances the trade-off between robustness and solution quality.

Result 2. The Monte Carlo policy and our ML-CO policy generalize well over uncertain future observations. The learning component in the ML-CO policy balances the trade-off between robustness and solution quality, leading to a superior performance.

6.5. Extended Analysis

This analysis assesses the impact of different training settings on the performance of our ML-CO policy. Specifically, we investigate the impact of (i) the feature set, (ii) the size of each training instance, (iii) the training set size, (iv) the imitated target strategy, and (v) the type of statistical model used. We evaluate each model's performance as detailed in Section 6.2 and report the relative gap to the anticipative baseline.

6.5.1. Different Feature Sets. Table 3 compares the performance of our ML-CO policy on three feature sets (i.e., *complete*, *model-aware*, and *model-free*). Table 4 details the features each set comprises. Specifically, the *model-free* feature set contains features computable from the current state x^e , whereas the *model-aware* feature set only contains features which include distributional information from the static instance. We further include the *complete* feature set, which combines the features from the *model-aware* and *model-free* feature sets. The *complete* feature set was used in the model submitted to the challenge, that is, our baseline. Our results show that the

Table 3. Performance of ML-CO Policy Using Different Feature Sets

	Feature sets		
	Complete (baseline)	Model-aware	Model-free
Relative distance to ant. b.	5.15%	13.83%	6.78%

Table 4. Different Feature Sets

Model-free		Model-aware	
x coordinate of location	x_r	Quantiles from distribution of travel time to all locations:	
y coordinate of location	y_r	1% quantile	$Pr[X < x] \leq 0.01, X \sim t_{r,:}$
demand	q_r	5% quantile	$Pr[X < x] \leq 0.05, X \sim t_{r,:}$
service time	s_r	10% quantile	$Pr[X < x] \leq 0.1, X \sim t_{r,:}$
time window start	l_r	50% quantile	$Pr[X < x] \leq 0.5, X \sim t_{r,:}$
time window end	u_r	Quantiles from distribution of slack time to all time windows:	
time from depot to request	$t_{d,r}$	0% quantile	$Pr[X < x] \leq 0, X \sim u, - (l_r + s_r + t_{r,:})$
relative time depot to request	$t_{d,r} / (u_r - s_r)$	1% quantile	$Pr[X < x] \leq 0.01, X \sim u, - (l_r + s_r + t_{r,:})$
time window start / rem. time	$l_r / (T_{max} - \tau_e)$	5% quantile	$Pr[X < x] \leq 0.05, X \sim u, - (l_r + s_r + t_{r,:})$
time window end / rem. time	$u_r / (T_{max} - \tau_e)$	10% quantile	$Pr[X < x] \leq 0.1, X \sim u, - (l_r + s_r + t_{r,:})$
is must dispatch	$\mathbb{1}_{\tau_e + \Delta\tau + t_{d,r} > u_r}$	50% quantile	$Pr[X < x] \leq 0.5, X \sim u, - (l_r + s_r + t_{r,:})$

Note. rem., remaining.

performance of our ML-CO policy does not rely solely on *model-aware* information not available in real-world scenarios. Specifically, considering *model-aware* features derived from the static instance decreases the gap of our ML-CO policy to the anticipative baseline by only 1.63 percentage points on average.

Result 3. Our ML-CO policy performs well without considering *model-aware* features derived from the static instance.

6.5.2. Different Sample Size of Training Instances. Table 5 shows the performance of the ML-CO policy relative to the performance of the anticipative baseline when training the ML-CO policy on training instances derived from different sample sizes. Note that we evaluate the trained models on instances generated with a sample size of 100 regardless of the sample size used during training. Our results indicate that there exists a trade-off between different sample sizes. Specifically, our pipeline performs best when training on instances with a sample size of 50, reaching an average gap of 5.15% to the anticipative baseline. Increasing or decreasing the sample size reduces the performance. It is not surprising to see the performance decrease when the sample size is small: The policy overfits the training set in that case and generalizes poorly. The decrease of performance on large sample sizes is more unexpected. It might be because we overfit the portion of the state space visited by the anticipative policy on the training instances, which may be different from those visited by our nonanticipative policy on the test instances.

Result 4. It is crucial to balance learning accuracy and model generalization when training the ML-CO policy.

6.5.3. Different Numbers of Training Instances. Table 6 shows the performance of the ML-CO policy relative to the performance of the anticipative baseline when training the ML-CO policy on different training set sizes. Recall that the policy is fitted on a set of training instances and tested on a test of different instances, and that for each instance, we solve 15 anticipative scenarios. The size of the training set therefore increases with the number of training instances. Our results convey that the performance of our ML-CO policy improves with the training set size. This is unsurprising, because first, a larger training set enables us to reduce single decisions in overfitting (independently of the number of instances), and second, a more diverse training set enables us to improve cross-instance generalization. The marginal improvement decreases with the training set size. Specifically, performance saturates at a training set size of 10 instances, which shows that our ML-CO policy learns to generalize well even from small training sets. This indicates that our approach requires only a few instances to extract most of the structural information contained in the problem setting, which is a stark contrast to findings from classic supervised learning, which generally requires significantly larger training sets.

Result 5. The ML-CO policy needs only a few training instances to learn a general policy.

6.5.4. Varying Target Strategies. To investigate the impact of the quality of the solutions imitated by our ML-CO policy, we vary the time limit allocated to the HGS used to derive the underlying anticipative solutions. Here the intuition is as follows: solutions derived with a low time limit should be of lower quality than solutions derived with a high time limit. As the HGS

Table 5. Performance of ML-CO Policy When Trained on Different-Sized Training Instances

	Sample size of training instances				
	10	25	50	75	100
Relative distance to ant. b.	8.71%	6.31%	5.15%	7.41%	13.08%

Table 6. Performance of ML-CO Policy When Training on Different Numbers of Training Instances

	No. of training instances							
	1	2	5	10	15	20	25	30
Relative distance to ant. b.	10.19%	7.77%	7.26%	5.89%	5.15%	5.23%	4.50%	4.94%

algorithm is subject to random decisions, we further include a training set derived from the best solutions found across 10 runs with a time limit of 3,600 seconds each. Table 7 shows the performance of the ML-CO policy relative to the performance of the anticipative baseline when training the ML-CO policy on different target solutions with different solution qualities. Surprisingly, there is no clear relation between the solution quality of the respective training set and the performance of the ML-CO policy. This leads to the assumption that an improved quality of the anticipative target solution does not increase the performance of the trained ML-CO policy and therefore the anticipative strategy might not be the best policy to imitate.

Result 6. The anticipative strategy might not be the best policy to imitate, and there might exist a target solution which yields better performances.

6.5.5. Different Statistical Models. Figure 5 compares the performance of our ML-CO policy when using different statistical models φ_w . All statistical models rely on a feature mapping ϕ that maps a state x^e and a request i of this state to a feature vector $\phi(i, x^e)$ in $\mathbb{R}^{|\phi|}$. Our *linear model* $\varphi_w(x^e) = (w^\top \phi(i, x))_i$ manages to handle variable-size inputs and outputs by applying the same linear model $\phi \mapsto w^\top \phi$ independently to each dimension i . Similarly, our *neural network* $\varphi_w(x^e) = (g_w(\phi(i, x)))_i$ applies an auxiliary neural network g_w independently for each dimension i . Finally, we propose two *graph neural networks*. Both graph neural networks consider requests as nodes and the connection between requests as edges. We include a regular and a sparsified graph neural network; the latter contains only those edges which are feasible with respect to request time windows and travel times. Both graph neural networks receive an input vector $\phi(i, x)$ for each node i , such that $\varphi_w(x^e) = (h_w(\phi(i, x)_{i \in I}))$.

The linear model performs worst with an average gap of 6.85% to the anticipative baseline whereas the sparsified graph neural network performs best with a 5.04% gap to the anticipative baseline. Comparing the

performance of the linear model to the neural networks' performance indicates the importance of a feature generator. In fact, using a simple neural network already lowers the gap to the anticipative baseline to 5.15%. As expected, using a graph neural network that calculates structural features further improves the performance of the ML-CO policy. However, the improvement is rather small in comparison with the performance of the neural network. This suggests that most of the structural information is already included in the structured learning approach.

Result 7. Feature-generating statistical models yield the best-performing ML-CO policies.

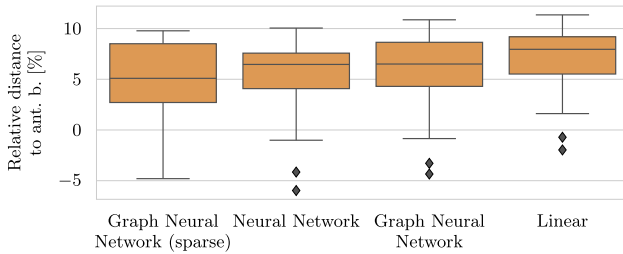
7. Conclusion

We presented a novel CO-enriched ML pipeline for a dynamic VRP that was introduced in the EURO Meets NeurIPS Vehicle Routing Competition. Specifically, our work contains several methodological contributions and an extensive computational study. From a methodological perspective, we extend ML-based pipelines to objective functions where the dimension of the predicted objective costs does not match the dimension of the decision variables. These objective functions can be adapted to other problem settings such that we have made them available in the open-source library `InferOpt.jl`. Moreover, we presented the first pipeline that utilizes a metaheuristic component to solve the CO layer and showed how to carefully design a metaheuristic that finds heuristic solutions which allow us to compute approximate gradients. We showed how to train the ML layer of this pipeline in a supervised learning fashion, that is, based on a training set derived from an anticipative strategy. We presented a comprehensive numerical study and show that our policy encoded via the CO-enriched ML pipeline outperforms greedy policies by 13.18% and even Monte Carlo policies, which were granted a longer runtime, by 1.57% in terms of travel time on average. Interestingly, our results point at the fact that counterintuitive to common practice,

Table 7. Performance of ML-CO Policy for Different Target Strategies

	Imitated anticipative strategies							
	Best seed	3,600 sec	900 sec	300 sec	240 sec	180 sec	120 sec	60 sec
Relative distance to ant. b.	6.18%	5.15%	4.79%	7.61%	6.01%	5.25%	5.33%	5.81%
Average objective value (×10,000)	20.49	20.56	20.70	20.78	20.80	20.87	20.97	21.21

Figure 5. (Color online) Performance of ML-CO Policy Using Different Statistical Models



imitating anticipative strategies can work well for high-dimensional multistage stochastic optimization problems, even if the anticipative strategy might not be the best strategy to imitate.

In this work, we limited the evaluation of the presented CO-enriched ML pipeline to the problem setting provided in the EURO Meets NeurIPS Vehicle Routing Competition. This problem setting was tailored to the competition and therefore had some shortcomings in comparison with general real-world practices, for example, an unlimited vehicle fleet and long epoch durations of one hour. To what extent the performance of our proposed CO-enriched ML pipeline and CO-enriched ML pipelines in general transfers to other problem settings hence remains an open question. Although some early works point in this direction (see, e.g., Jungel et al. 2023), answering this question requires future research on the added value of these pipelines in the context of more general dynamic problem settings, for example, in the areas of mobility-on-demand, freight and food delivery, and inventory management. Furthermore, although the use of CO-enriched ML to encode policies seems very promising, stochastic optimization experience and artifacts in our results indicate that imitating the anticipative strategy might not be the best learning approach. Improving the learning algorithm may therefore be a fruitful research direction. Contrarily, if imitating a “bad” anticipative policy does work on other applications, it would be interesting to understand this phenomenon through a theoretical lens. Finally, we believe that exploring applications with shorter, possibly fully dynamic epoch durations and problem settings that allow dynamic adaption of routes after initial dispatch could reveal areas where CO-enriched ML pipelines provide added value over existing approaches, and thus lead to a more profound understanding of when CO-enriched ML pipelines in general perform well.

Acknowledgments

The authors thank Wouter Kool and the organizational committee of the EURO Meets NeurIPS 2022 Vehicle Routing Competition for providing the problem setting studied in this paper. Léo Baty, Kai Jungel, and Patrick S. Klein made equal contributions.

References

- Akkerman F, Luy J, van Heeswijk W, Schiffer M (2023) Handling large discrete action spaces via dynamic neighborhood construction. Preprint, submitted May 31, <https://arxiv.org/abs/2305.19891v1>.
- Alinaghian M, Aghaie M, Sabbagh MS (2019) A mathematical model for location of temporary relief centers and dynamic routing of aerial rescue vehicles. *Comput. Indust. Engrg.* 131:227–241.
- Alonso-Mora J, Wallar A, Rus D (2017) Predictive routing for autonomous mobility-on-demand systems with ride-sharing. 2017 *IEEE/RSJ Internat. Conf. Intelligent Robots Systems (IROS)* (IEEE, Piscataway, NJ), 3583–3590.
- Basso R, Kulcsár B, Sanchez-Diaz I, Qu X (2022) Dynamic stochastic electric vehicle routing with safe reinforcement learning. *Transportation Res. Part E Logist. Transportation Rev.* 157:102496.
- Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: A methodological tour d’horizon. *Eur. J. Oper. Res.* 290(2):405–421.
- Bent RW, Van Hentenryck P (2004) Scenario-based planning for partially dynamic vehicle routing with stochastic customers. *Oper. Res.* 52(6):977–987.
- Berthet Q, Blondel M, Teboul O, Cuturi M, Vert JP, Bach F (2020) Learning with differentiable perturbed optimizers. *Adv. Neural Inform. Processing Systems* 33 (Curran Associates, Inc., Red Hook, NY), 9508–9519.
- Blondel M, Martins AF, Niculae V (2020) Learning with Fenchel-Young losses. *J. Machine Learn. Res.* 21(1):1314–1382.
- Carpentier P, Chancelier JP, Cohen G, De Lara M (2015) *Stochastic Multi-stage Optimization*, Probability Theory and Stochastic Modeling, vol. 75 (Springer, Cham, Switzerland).
- Chu H, Zhang W, Bai P, Chen Y (2023) Data-driven optimization for last-mile delivery. *Complex Intelligent Systems* 9:2271–2284.
- Dalle G, Baty L, Bouvier L, Parmentier A (2022) Learning with combinatorial optimization layers: A probabilistic approach. Preprint, submitted July 27, <https://arxiv.org/abs/2207.13513>.
- Elmachtoub AN, Grigas P (2022) Smart “predict, then optimize.” *Management Sci.* 68(1):9–22.
- Elmachtoub AN, Liang JCN, McNellis R (2020) Decision trees for decision-making under the predict-then-optimize framework. *Proc. Machine Learn. Res.* 119:2858–2867.
- Enders T, Harrison J, Pavone M, Schiffer M (2023) Hybrid multi-agent deep reinforcement learning for autonomous mobility on demand systems. *Proc. Machine Learn. Res.* 211:1–13.
- Fikar C (2018) A decision support system to investigate food losses in e-grocery deliveries. *Comput. Indust. Engrg.* 117:282–290.
- Flatberg T, Hasle G, Kloster O, Nilssen EJ, Riise A (2007) Dynamic and stochastic vehicle routing in practice. Zempeks V, Tarantilis CD, Giaglis GM, Minis I, eds. *Dynamic Fleet Management*, Operations Research/Computer Science Interfaces Series, vol. 38 (Springer, Boston), 41–63.
- Gendreau M, Guertin F, Potvin JY, Taillard E (1999) Parallel tabu search for real-time vehicle routing and dispatching. *Transportation Sci.* 33(4):381–390.
- Hildebrandt FD, Thomas BW, Ulmer MW (2023) Opportunities for reinforcement learning in stochastic dynamic vehicle routing. *Comput. Oper. Res.* 150:106071.
- Joe W, Lau HC (2020) Deep reinforcement learning approach to solve dynamic vehicle routing problem with stochastic customers. *Proc. Internat. Conf. Automated Planning Scheduling* (AAAI Press, Palo Alto, CA), 394–402.
- Jungel K, Parmentier A, Schiffer M, Vidal T (2023) Learning-based online optimization for autonomous mobility-on-demand fleet control. Preprint, submitted February 8, <https://arxiv.org/abs/2302.03963>.
- Kool W, van Hoof H, Welling M (2019) Attention, learn to solve routing problems! *Internat. Conf. Learn. Representations (ICLR, Appleton, WI)*.

- Kool W, Juninck JO, Roos E, Cornelissen K, Agterberg P, van Hoorn J, Visser T (2022a) Hybrid genetic search for the vehicle routing problem with time windows: A high-performance implementation. *12th DIMACS Implementation Challenge Workshop*.
- Kool W, Blik L, Numeroso D, Reijnen R, Afshar RR, Zhang Y, Catshoek T, et al. (2022b) The EURO Meets NeurIPS 2022 Vehicle Routing Competition. <https://neurips.cc/virtual/2022/competition/50085>.
- Kotary J, Fioretto F, Van Hentenryck P, Wilder B (2021) End-to-end constrained optimization learning: A survey. Preprint, submitted March 30, <https://arxiv.org/abs/2103.16378>.
- Liang E, Wen K, Lam WHK, Sumalee A, Zhong R (2022) An integrated reinforcement learning and centralized programming approach for online taxi dispatching. *IEEE Trans. Neural Networks Learn. Systems* 33(9):4742–4756.
- Liu S, He L, Shen ZJM (2021) On-time last-mile delivery: Order assignment with travel-time predictors. *Management Sci.* 67(7):4095–4119.
- Nagata Y, Kobayashi S (2010) A memetic algorithm for the pickup and delivery problem with time windows using selective route exchange crossover. Schaefer R, Cotta C, Kołodziej J, Rudolph G, eds. *Parallel Problem Solving from Nature, PPSN XI. PPSN 2010*, Lecture Notes in Computer Science, vol. 6238 (Springer, Berlin), 536–545.
- Nazari M, Oroojlooy A, Snyder L, Takac M (2018) Reinforcement learning for solving the vehicle routing problem. Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Gamett R, eds. *Adv. Neural Inform. Processing Systems 31* (Curran Associates, Inc., Red Hook, NY).
- Nicolai B (2016) Amazon liefert pakete innerhalb von zwei stunden. *Welt* (March 2016), <https://www.welt.de/wirtschaft/article153690106/Amazon-liefert-Pakete-innerhalb-von-zwei-Stunden.html>.
- Ojeda Rios BH, Xavier EC, Miyazawa FK, Amorim P, Curcio E, Santos MJ (2021) Recent dynamic vehicle routing problems: A survey. *Comput. Indust. Engrg.* 160:107604.
- Oliver I, Smith D, Holland JR (1987) A study of permutation crossover operators on the traveling salesman problem. Grefenstette JJ, ed. *Genetic Algorithms and Their Applications* (Psychology Press, New York), 224–230.
- Parmentier A (2021) Learning structured approximations of operations research problems. Preprint, submitted July 9, <https://arxiv.org/abs/2107.04323>.
- Parmentier A (2022) Learning to approximate industrial problems by operations research classic problems. *Oper. Res.* 70(1):606–623.
- Parmentier A, T'Kindt V (2021) Learning to solve the single machine scheduling problem with release times and sum of completion times. Preprint, submitted January 4, <https://arxiv.org/abs/2101.01082>.
- Pflug GC, Pichler A (2014) *Multistage Stochastic Optimization*, Springer Series in Operations Research and Financial Engineering (Springer, Cham, Switzerland).
- Pillac V, Gendreau M, Guéret C, Medaglia AL (2013) A review of dynamic vehicle routing problems. *Eur. J. Oper. Res.* 225(1):1–11.
- Placek M (2022) Same-day delivery market size in U.S. 2019–2024. <https://www.statista.com/statistics/1068886/us-same-day-delivery-market-size/>.
- Raza SM, Sajid M, Singh J (2022) Vehicle routing problem using reinforcement learning: Recent advancements. Gupta D, Sambyo K, Prasad M, Agarwal S, eds. *Advanced Machine Intelligence and Signal Processing* (Springer Nature Singapore, Singapore), 269–280.
- Ritzinger U, Puchinger J, Hartl RF (2016) A survey on dynamic and stochastic vehicle routing problems. *Internat. J. Production Res.* 54(1):215–231.
- Soeffker N, Ulmer MW, Mattfeld DC (2022) Stochastic dynamic vehicle routing in the light of prescriptive analytics: A review. *Eur. J. Oper. Res.* 298(3):801–820.
- Steever Z, Karwan M, Murray C (2019) Dynamic courier routing for a food delivery service. *Comput. Oper. Res.* 107:173–188.
- Tang X, Qin Z, Zhang F, Wang Z, Xu Z, Ma Y, Zhu H, Ye J (2019) A deep value-network based approach for multi-driver order dispatching. *Proc. 25th ACM SIGKDD Internat. Conf. Knowledge Discovery Data Mining* (ACM, New York), 1780–1790.
- Ulmer MW, Soeffker N, Mattfeld DC (2018) Value function approximation for dynamic multi-period vehicle routing. *Eur. J. Oper. Res.* 269(3):883–899.
- van Doorn J, Lan L, Pentinga L, Wouda N (2022) Solving a static and dynamic VRP with time windows using hybrid genetic search and simulation. EURO Meets NeurIPS Vehicle Routing Competition, <https://github.com/ortec/euro-neurips-vrp-2022-quickstart/blob/main/papers/OptiML.pdf>.
- Vidal T (2022) Hybrid genetic search for the CVRP: Open-source implementation and swap* neighborhood. *Comput. Oper. Res.* 140:105643.
- Vidal T, Laporte G, Matl P (2020) A concise guide to existing and emerging vehicle routing problem variants. *Eur. J. Oper. Res.* 286(2):401–416.
- Vidal T, Maculan N, Ochi LS, Vaz Penna PH (2016) Large neighborhoods with implicit customer selection for vehicle routing problems with profits. *Transportation Sci.* 50(2):720–734.
- Waller A, Van Der Zee M, Alonso-Mora J, Rus D (2018) Vehicle rebalancing for mobility-on-demand systems with ride-sharing. *2018 IEEE/RSJ Internat. Conf. Intelligent Robots Systems (IROS)* (IEEE, Piscataway, NJ), 4539–4546.
- Zhou M, Jin J, Zhang W, Qin Z, Jiao Y, Wang C, Wu G, Yu Y, Ye J (2019) Multi-agent reinforcement learning for order-dispatching via order-vehicle distribution matching. *Proc. 28th ACM Internat. Conf. Inform. Knowledge Management* (Association for Computing Machinery, New York), 2645–2653.